



TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY



VIỆN TOÁN ỨNG
DỤNG VÀ TIN HỌC
SCHOOL OF APPLIED
MATHEMATICS AND INFORMATICS

PHÂN LOẠI CẢM XÚC ĐA MỨC TRÊN AMAZON REVIEWS

Mô hình BiLSTM kết hợp Additive Attention

Nhóm 5
2026

ONE LOVE. ONE FUTURE

1. GVHD: PGS. TS. Ngô Văn Linh

2. Trần Lê Ngọc Tâm

MSSV: 202416346

3. Nguyễn Phong

MSSV: 202400066

4. Nguyễn Ngọc Tuấn Anh

MSSV: 202400029

1. Bài toán và dữ liệu
2. BiLSTM kết hợp Attention
3. So sánh mô hình
4. Kết luận
5. Tài liệu tham khảo

Bài toán

Phân loại mỗi review vào **một trong 5 lớp**:

$$y \in \{1, 2, 3, 4, 5\} \text{ sao.}$$

Đây là **multiclass classification**, không phải hồi quy.

- ◇ Đầu ra: Softmax 5 lớp.
- ◇ Loss: sparse categorical cross-entropy.
- ◇ Dự đoán: lớp có xác suất lớn nhất.

Mục tiêu chính

- ◇ Nắm bắt ngữ cảnh hai chiều.
- ◇ Tập trung vào token quan trọng.
- ◇ So sánh với SimpleRNN và Transformer Encoder.
- ◇ Đánh giá cả lớp thiếu số, không chỉ Accuracy.

Nguồn dữ liệu

Amazon Reviews Dataset

Mở trang dữ liệu trên Kaggle

- ◇ Hai trường sử dụng: Review Text, Rating.
- ◇ Sau làm sạch: **20.405 review hợp lệ**.
- ◇ Chia phân tầng: 80% train, 20% validation.
- ◇ Validation: **4.081 mẫu**.

Baseline dễ gây hiểu nhầm

Nếu luôn đoán **1 sao**, mô hình đã đạt **63,49% Accuracy** mà chưa học được năm

Lớp	Số mẫu	Tỷ lệ
1 sao	2.591	63,49%
2 sao	241	5,91%
3 sao	165	4,04%
4 sao	239	5,86%
5 sao	845	20,71%

Lớp thiểu số

Ba lớp 2–4 sao chỉ chiếm **15,80%**. Vì vậy kết quả phải đọc cùng **Macro F1, rating MAE** và F1 từng lớp.

Macro F1: công bằng giữa các lớp

MAE: đo độ xa của lỗi rating

$$F1_k = \frac{2P_kR_k}{P_k + R_k},$$

$$\text{MacroF1} = \frac{1}{5} \sum_{k=1}^5 F1_k.$$

$$\text{MAE}_{\text{rating}} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|.$$

1 → 2: lỗi 1 1 → 5: lỗi 4

Mỗi lớp đóng góp đúng **20%**, bất kể có 2.591 hay chỉ 165 mẫu. Nếu bỏ qua lớp 2–4 sao, Macro F1 giảm mạnh.

Ba góc nhìn bổ sung cho nhau

Accuracy: đúng hay sai

Macro F1: có bỏ quên lớp thiểu số không

MAE:



Một review đi qua pipeline

Dữ liệu thô Battery is NOT good!!! Rating: 2

Sau làm sạch battery is not good

Token IDs [842, 17, 39, 64] (minh họa)

Sau padding [842, 17, 39, 64, 0, 0, ..., 0] $\in \mathbb{N}^{200}$

Embedding $X \in \mathbb{R}^{200 \times 100}$; mỗi ID được ánh xạ sang vector GloVe 100 chiều.

Chống leakage: chia dữ liệu trước, rồi mới học vocabulary từ train.

GloVe cung cấp gì?

- ◇ Học từ thống kê đồng xuất trên **6 tỷ token** của Wikipedia 2014 + Gigaword 5.
- ◇ Khoảng **400.000 từ**; có sẵn các bản 50/100/200/300 chiều.
- ◇ Các từ gần nghĩa có vector gần nhau ngay trước khi train trên Amazon Reviews.

Phù hợp với thí nghiệm này

- ◇ Tập train chỉ có **16.324 review**: học embedding hoàn toàn từ đầu để thiếu dữ liệu.
- ◇ **100d** cân bằng thông tin và chi phí: $26.715 \times 100 \approx 2,67$ triệu tham số.
- ◇ `trainable=True`: giữ tri thức pretrained, sau đó thích nghi với ngôn ngữ review.

Trade-off

Nhanh và nhẹ hơn 300d, giàu thông tin hơn 50d; nhưng không phải embedding theo ngữ cảnh và có thể thiếu từ đặc thù sản phẩm.

LSTM

Ghi nhớ phụ thuộc dài tốt hơn SimpleRNN nhờ cell state và các cổng.

Bidirectional

Mỗi token nhận ngữ cảnh từ cả bên trái và bên phải.

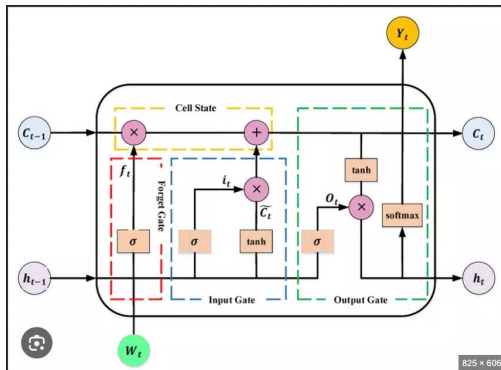
Attention

Gán trọng số lớn hơn cho token quan trọng đối với sentiment.

Ví dụ

“The screen is excellent, **but** the battery is **disappointing**.”

Mô hình cần đọc hai chiều và xác định phần nào ảnh hưởng mạnh hơn tới rating cuối cùng.



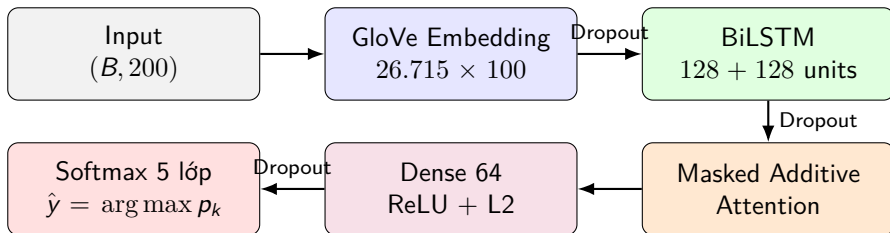
Cell state c_t là “đường nhớ”; ba cổng điều khiển luồng thông tin.

$$\begin{aligned} f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\ i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), \\ \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), \\ h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

f_t : quên thông tin cũ; i_t : ghi thông tin mới; o_t : chọn phần được xuất.

Ý nghĩa

Phép cộng trên c_t tạo đường truyền gradient ổn định hơn SimpleRNN, giúp giữ tín hiệu sentiment qua chuỗi dài.



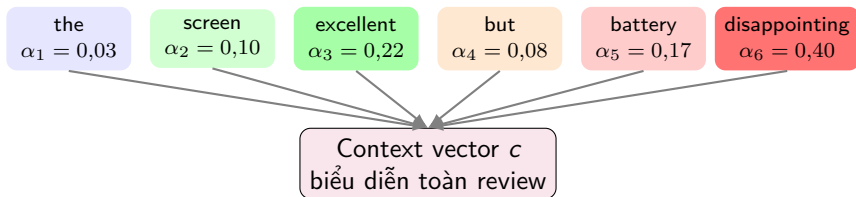
◇ $h_t = [\vec{h}_t; \overleftarrow{h}_t] \in \mathbb{R}^{256}$.

◇ BiLSTM output: $(B, 200, 256)$.

◇ Attention context: $(B, 256)$.

◇ Dropout: 0,5; L2: 0,01.

◇ Tổng tham số: 2.988.817.



Không dùng hidden state cuối

BiLSTM trả về $H = [h_1, \dots, h_{200}]$, với $h_t \in \mathbb{R}^{256}$. Attention học mức đóng góp của **từng vị trí**.

Các trọng số trên chỉ là ví dụ minh họa cơ chế, không phải output đã trích từ artifact hiện tại.

Trực giác

Token mang sentiment mạnh nhận α_t lớn. Context là trung bình có trọng số, nên vẫn giữ thông tin ở xa cuối câu.

Với $H \in \mathbb{R}^{B \times 200 \times 256}$.

$$u_t = \tanh(W_a h_t + b_a),$$

$$e_t = v_a^\top u_t,$$

$$\alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^T \exp(e_j)},$$

$$c = \sum_{t=1}^T \alpha_t h_t.$$

Vai trò của từng hàm

- ◇ $\tanh$: tạo phép chấm điểm phi tuyến.
- ◇ $v_a^\top$: nén vector thành một score vô hướng.
- ◇ Softmax: chuẩn hóa $\alpha_t \geq 0$ và $\sum_t \alpha_t = 1$.

Mask padding

Trước Softmax:

$$e_t \leftarrow \begin{cases} e_t, & m_t = 1, \\ -10^9, & m_t = 0. \end{cases}$$

Vì $\exp(-10^9) \approx 0$, token 0 không được góp vào context.

Tham số học được

- ◇ $W_a \in \mathbb{R}^{256 \times 256}$
- ◇ $b_a \in \mathbb{R}^{256}$
- ◇ $v_a \in \mathbb{R}^{256}$

Tổng: **66.048** tham số.

1. Khớp nhãn bằng Cross-Entropy

Softmax đổi logits thành phân phối xác suất:

$$p_{i,k} = \frac{\exp(z_{i,k})}{\sum_{j=0}^4 \exp(z_{i,j})}, \quad \sum_{k=0}^4 p_{i,k} = 1.$$

Với nhãn nguyên $y_i \in \{0, \dots, 4\}$:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \log p_{i,y_i}.$$

Bản **sparse** nhận trực tiếp nhãn nguyên, không cần one-hot.

Tại sao “chồng” hai thành phần?

- ◇ Softmax + CE: tạo xác suất rồi tối đa hóa xác suất của lớp đúng; sai mà quá tự tin sẽ bị phạt mạnh.
- ◇ L2: hạn chế trọng số quá lớn, giảm overfitting.
- ◇ Một thành phần học dữ liệu; một thành phần kiểm soát độ phức tạp.

Nhược điểm trong bài toán này

2. Phạt độ phức tạp bằng L2

Khi huấn luyện

$$\mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{L2}}$$

Trơn, khả vi và cho gradient trực tiếp tới toàn bộ xác suất Softmax. Adam tối thiểu hóa loss này.

Cấu hình

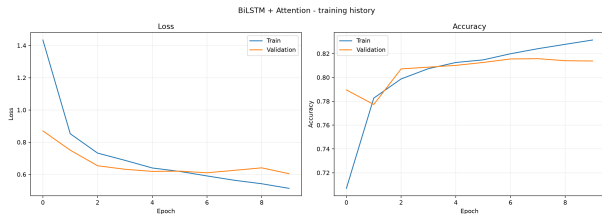
Adam 10^{-3} ; batch 128; tối đa 10 epoch;
Dropout 0,5; EarlyStopping theo `val_loss`.

Khi đánh giá

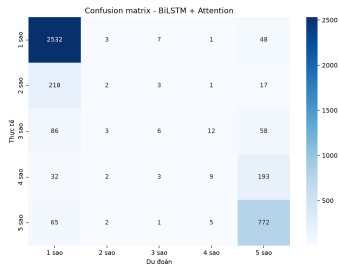
- ◇ **Macro F1**: đo chất lượng đồng đều trên năm lớp.
- ◇ **Rating MAE**: đo khoảng cách giữa hai rating.
- ◇ Cả hai đang tính sau `arg max`, nên không thuận tiện để tối ưu trực tiếp bằng gradient.

Hướng cải thiện

Class-weighted CE hoặc focal loss cho lớp thiểu số; ordinal loss hoặc CE kết hợp expected-rating loss để đưa thứ tự 1–5 vào quá trình học



Learning curve trên 10 epoch

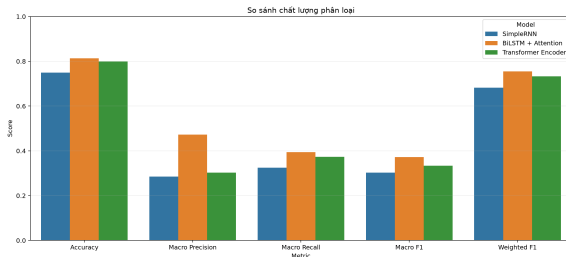


Confusion matrix trên 4.081 mẫu validation

Accuracy 81,38%

Macro F1 0,3727

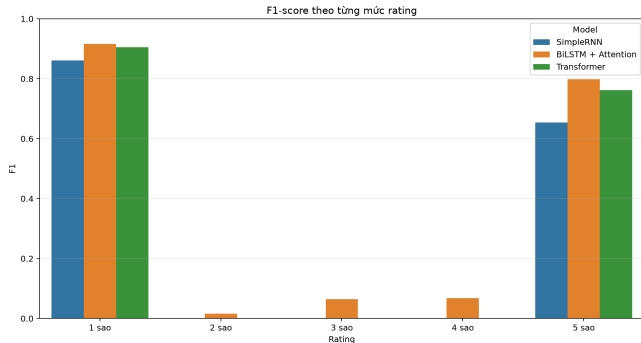
Rating MAE 0,3328



Mô hình	Acc.	Macro F1	MAE
SimpleRNN	0,7493	0,3029	0,6288
BiLSTM + Attn.	0,8138	0,3727	0,3328
Transformer	0,7988	0,3336	0,4009

Nhận xét

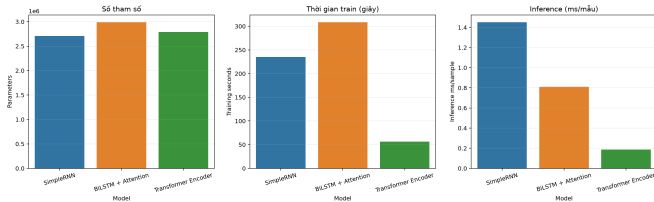
- ◇ BiLSTM + Attention tốt nhất ở mọi metric chất lượng.
- ◇ Hơn Transformer 1,49 điểm % Accuracy.
- ◇ Giảm MAE 47,08% so với SimpleRNN.



Phát hiện quan trọng

- ◇ Cả ba mô hình nhận diện tốt lớp 1 và 5 sao.
- ◇ SimpleRNN và Transformer có F1 bằng 0 ở lớp 2–4.
- ◇ BiLSTM + Attention có dự đoán đúng lớp giữa, nhưng F1 vẫn dưới 0,07.

Accuracy cao **không đồng nghĩa** bài toán 5 lớp đã được giải quyết tốt.



Đánh đổi

- ◇ BiLSTM + Attention đạt chất lượng cao nhất nhưng train lâu nhất: 308,47 giây.
- ◇ Transformer nhanh nhất: 56,42 giây train và 0,186 ms/mẫu.
- ◇ Số tham số ba mô hình khá gần nhau do embedding chiếm phần lớn.

Kết luận

- ◇ BiLSTM + Attention là mô hình tốt nhất trong ba kiến trúc đã thử.
- ◇ BiLSTM cung cấp ngữ cảnh hai chiều; attention tổng hợp token linh hoạt hơn hidden state cuối.
- ◇ Mô hình phù hợp làm kiến trúc chính của project.

Hạn chế và hướng tiếp theo

- ◇ Mất cân bằng lớp làm mô hình co cụm về 1 và 5 sao.
- ◇ Bật class weights hoặc thử focal loss.
- ◇ Chọn checkpoint theo Macro F1.
- ◇ Thử ordinal loss và tạo test set độc lập.
- ◇ Xuất attention weights để phân tích token.

1. Amazon Reviews Dataset, Kaggle:
<https://www.kaggle.com/dongrelaxman/amazon-reviews-dataset>.
2. S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997.
3. D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," *ICLR*, 2015.
4. J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global Vectors for Word Representation," *EMNLP*, 2014.
5. A. Vaswani et al., "Attention Is All You Need," *NeurIPS*, 2017.

**Cảm ơn mọi người
đã chú ý lắng nghe!**